

Actividad: De lo lineal a lo jerárquico

Estructura de Datos II · Ingeniería de Software / Sistemas · Semestre IV
Estudiante: Yullianth David Montoya Montero

1. Clasificación de los cuatro escenarios

Escenario	Estructura	Justificación
Sistema de archivos	Jerárquica	Las carpetas contienen archivos y otras carpetas; cada elemento tiene un único padre y una ruta única desde la raíz.
Organigrama empresarial	Jerárquica	Existe una raíz (la empresa) y cada área depende de una sola unidad superior.
Menú de una aplicación	Jerárquica	Menús, submenús y opciones forman varios niveles. Sería lineal solo si fuera una barra sin submenús.
Árbol genealógico	Jerárquica (con matices)	Modela generaciones padre-hijo; como cada persona tiene dos progenitores y hay uniones entre familias, se acerca a un grafo. Es árbol si se sigue una sola línea.

2. Caso elegido: organigrama empresarial

El diagrama elaborado en Draw.io representa una empresa con **12 nodos y 4 niveles**. La raíz es *Empresa*; en el segundo nivel están Tecnología, Finanzas y Recursos Humanos; en el tercero, sus áreas (Desarrollo, Infraestructura, Contabilidad, Tesorería, Selección y Bienestar); y en el cuarto, Backend y Frontend, que dependen de Desarrollo. Las **hojas** son los nodos sin dependientes directos (7 en total), y cada flecha indica la relación padre-hijo.

3. ¿Por qué no basta una estructura lineal?

Una lista o un arreglo guarda los elementos en secuencia: permite saber *qué* áreas existen, pero no *de quién dependen*. En la versión lineal [Empresa, Tecnología, Desarrollo, Backend, ...] no se aprecia que Backend pertenece a Desarrollo ni que este pertenece a Tecnología. Además, agregar un nuevo departamento exigiría recorrer y reorganizar la lista. El árbol conserva la relación de dependencia, admite varios niveles y crece simplemente agregando hijos a un nodo.

4. Implementación en Java

La clase genérica `NodoGeneral<T>` guarda un dato y una lista de hijos. El método `agregarHijo()` crea la relación padre-hijo y `esHoja()` identifica nodos sin descendientes. Los recorridos son **recursivos**: `mostrarArbol()` imprime la jerarquía con sangría por nivel, y `contarNodos()`, `altura()` y `contarHojas()` consultan toda la estructura. El programa imprime los 12 nodos, la altura 4 y una comparación con la lista plana.

5. Conclusión

Cuando los datos tienen una relación de pertenencia (padre-hijo), la estructura jerárquica es más natural, más expresiva y más fácil de ampliar que una lineal. Las listas siguen siendo útiles para colecciones planas, pero no para representar dependencias entre niveles.